NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 11.
AI agent for REST API integration: Fetch API, Promise and async/await, API documentation analysis, loading and error states, debugging via Network

Modern Web Development: HTML, CSS, JS/TS

Deadline: 22.10.2026

1. Aim

Learn how to integrate a REST API into a frontend application based on its documentation: dispatch requests using the Fetch API, process responses and errors with `async/await`, abort stale requests, and present loading, empty, and error UI states to the user. Master auditing asynchronous code written by an AI agent against a specification, self-check tests, and the DevTools Network panel, rather than relying on "it works on my network connection."

Expected outcome: a working ShowFinder application powered by the live TVmaze API, where each feature was drafted by an AI agent and audited and fixed by you; "Passed: 24 of 24" in the self-check suite; an API reference summary `API.md` and an agent instructions file `AGENTS.md` in the repository.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- Dispatch requests using `fetch`, inspect `status`, `ok`, headers, and response bodies; explain why `fetch` does not reject its promise on 404 or 500 status codes.
- Write asynchronous code with `async/await`: handle errors with `try/catch`, execute independent requests concurrently (`Promise.all`), and distinguish `return` from `return await`.

- Abort outdated requests using `AbortController` and distinguish cancellations from runtime errors.
- Explain HTTP status codes 200, 301, 404, 429, and 5xx and what each means for the UI; the `Content-Type` and `Cache-Control` headers; the browser HTTP cache.
- Explain CORS: what the browser blocks, when preflight requests are triggered, and why CORS errors cannot be resolved client-side.
- Inspect network traffic using DevTools Network: status, headers, payload, initiator, timing, throttling, and request blocking.
- Read API documentation critically: endpoints, parameters, response schema, nullable fields, error responses, caching policies, and rate limits.
- Audit AI-generated asynchronous code and supply project context to an agent via a rules file.

3. Lab tasks

1. Carry out the research tasks (§4), conduct the experiments, and compile the `API.md` notes (§4.5).
2. Download the ShowFinder starter project, commit it to Git unchanged, and confirm the self-check reports "Passed: 0 of 24" (§5.1).
3. Complete the assignment, §4: Challenges 1–2, the `AGENTS.md` rules file, and Challenge 3. For each challenge, prompt the AI agent for a draft, commit it, audit against the assignment, and fix defects by hand (§5).
4. Run through the Network checklist (assignment, §6) on your final version, deploy the project to GitHub Pages or Vercel, and prepare your report (§6). Then proceed to oral defense (§7).

4. Research tasks

General reference: Using the Fetch API — MDN, HTTP — MDN, and Network features reference — Chrome DevTools; the API you are integrating is documented at TVmaze API.

These experiments examine browser and TVmaze API behavior, not your application code: you do not need to debug or fix them. The "Where this occurs" note in each experiment highlights where this behavior causes bugs in ShowFinder if overlooked.

Experiments scratchpad: The starter project (showfinder-starter.zip, same as in §5) includes an empty `experiments.html` page. Its only purpose is to provide a real origin so the browser applies CORS rules. Unpack the archive, start Live Preview for `experiments.html`, and open it in Chrome or Edge at `http://127.0.0.1:…` rather than in an embedded VS Code tab.

In DevTools (F12, on macOS Cmd+Option+I):

- **Network:** enable **Preserve log**, select the **Fetch/XHR** filter, and verify that **Disable cache** is unchecked (needed for Experiment 8);
- **Console:** paste code snippets in their entirety. The DevTools Console allows top-level `await` without wrapping in an `async` function.

Refresh the page and clear both Console and Network before each experiment. The snippets send live requests to TVmaze. Do not run them in tight loops: TVmaze allows at least 20 requests per 10 seconds per IP address.

Experiment protocol:

1. Read the description, **analyze** the code, and write down your expected result — answering the "Analyze" prompt.
2. Refresh the page and paste the snippet into the Console. Inspect DevTools Network whenever the prompt mentions network behavior.
3. If your predicted result differs from the actual outcome, explain where your reasoning was flawed.

The "Where this occurs" questions prepare you for defense; you do not need to answer them in writing. Summarize results in a table for your report:

#	Topic	My analysis	Actual result (Console and Network)	Explanation (if different)
1	fetch and 404 status	...	...	...

4.1. Fetch API and Promise

Study:

- Using the Fetch API, Response — MDN
- async function, Using promises — MDN
- Promise.all(), Promise.allSettled() — MDN
- AbortController — MDN

Investigate: what `fetch` returns and when its promise rejects; `ok` and `status`; why response bodies can only be consumed once; error propagation in `async` functions; parallel vs. sequential requests; how to abort requests and what the calling code receives.

Experiments:

1. **fetch and 404 status.** Queries a non-existent show.

```
const response = await fetch('https://api.tvmaze.com/shows/999999999');
console.log(response.ok, response.status);
console.log(await response.json());
```

Analyze: will `fetch` reject its promise? What will both `console.log` calls output? What appears in red in the Console, and does that mean the code crashed?
Where this occurs: search and show details (Challenge 3): a 404 or 500 response not checked via `response.ok` is mistakenly treated as valid data.

2. **Consuming response body once.** Reads the same response body twice, then attempts to parse an HTML document as JSON.

```
const response = await fetch('https://api.tvmaze.com/shows/82');
console.log((await response.json()).name);
try {
  await response.json();
} catch (error) {
  console.log('second time:', error.name);
}
const page = await fetch('experiments.html');
try {
  await page.json();
} catch (error) {
  console.log('HTML:', error.name, error.message);
}
```

Analyze: what will each `console.log` output? Why does the second `json()` call fail to return data? What error does the HTML produce, and what does the < character have to do with it?
Where this occurs: Challenge 3. Proxies, captive Wi-Fi portals, or server error pages return HTML with status 200. Furthermore, consuming the body for logging breaks subsequent data handling.

3. **forEach, for...of, and Promise.all.** Three cast requests executed three different ways.

`cache: 'no-store'` disables browser HTTP caching so every call hits the network.

```
const ids = [82, 139, 33320];
const load = async (id) => (await
fetch(`https://api.tvmaze.com/shows/${id}/cast`, { cache: 'no-store'
})).json();

const counts = [];
ids.forEach(async (id) => counts.push((await load(id)).length));
console.log('forEach:', counts.length);
await new Promise((resolve) => setTimeout(resolve, 2000));
console.log('after 2s:', counts.length);

let start = performance.now();
for (const id of ids) await load(id);
console.log('for…of:', Math.round(performance.now() - start), 'ms');

start = performance.now();
await Promise.all(ids.map(load));
console.log('Promise.all:', Math.round(performance.now() - start),
'ms');
```

Analyze: what will `forEach:` and `after 2s:` output? Which of the last two approaches is faster, and roughly by what factor? How do these requests appear in the Network Waterfall column?
Where this occurs: show details page (Challenge 2). Fetching episodes and cast sequentially adds cumulative delays. With `forEach` and `async`, loading states complete prematurely before data arrives.

4. **return vs. return await in try blocks.** The `.invalid` domain is guaranteed not to resolve, so the request cannot reach a server.

```
async function load(url) {
  try {
    return fetch(url);
  } catch (error) {
    console.log('catch in load:', error.name);
    return null;
  }
}
const result = await
load('https://api.tvmaze.invalid/shows/82').catch((error) => `outside
load: ${error.name}`);
console.log(result);
```

Analyze: what does the Console display — `catch in load: …` or `outside load: …`? Why? What changes if written as `return await fetch(url);`?
Where this occurs: Challenge 3: `try/catch` wrapping `return fetch(…)` fails to catch network rejections, resulting in `Uncaught (in promise)`.

5. **Promise.all vs. Promise.allSettled.** Three requests where one cannot reach a host.

```
const urls = ['https://api.tvmaze.com/shows/82',
'https://api.tvmaze.invalid/shows/1',
'https://api.tvmaze.com/shows/139'];
try {
  const responses = await Promise.all(urls.map((url) => fetch(url)));
  console.log('all:', responses.map((response) => response.status));
} catch (error) {
  console.log('all:', error.name);
}
const settled = await Promise.allSettled(urls.map((url) => fetch(url)));
console.log('allSettled:', settled.map((result) => result.status));
```

   Analyze: what will `all:` and `allSettled:` output? What happened to the two successful responses in the first case?
   Where this occurs: show page with concurrent requests (Challenge 2): if one request fails, `Promise.all` discards the rest. When is this desired, and when is it harmful?
6. **AbortController.** The first request is aborted before response headers arrive; the second is aborted after headers arrive, but before reading the body.

```
const early = new AbortController();
const request = fetch('https://api.tvmaze.com/search/shows?q=girls', {
signal: early.signal });
early.abort();
try {
  await request;
} catch (error) {
  console.log('before response:', error.name);
}
const late = new AbortController();
const response = await fetch('https://api.tvmaze.com/shows/139', {
signal: late.signal });
late.abort();
console.log('after response:', response.status);
try {
  await response.json();
} catch (error) {
  console.log('json():', error.name);
}
```

   Analyze: what will each line output? What status appears in DevTools Network for aborted requests?
   Where this occurs: typing a new query aborts the prior search (Challenge 1); picking another show aborts previous details (Challenge 2). Abort is not an application error and must not display error notifications (Challenge 3).

4.2. HTTP: status codes, headers, caching

Study:

- HTTP response status codes: 200, 301, 404, 429, 500, 503 — MDN
- Content-Type, Cache-Control, HTTP caching — MDN
- CORS-safelisted response header — MDN
- URLSearchParams, encodeURIComponent() — MDN

Investigate: status code categories and their UI meaning; idempotent `GET`; accessible response headers in scripts; how `Cache-Control: max-age` governs browser cache; encoding query string parameters.

Experiments:

7. **Response headers.**

   ```
   const response = await fetch('https://api.tvmaze.com/shows/82');
   console.log(response.headers.get('content-type'));
   console.log(response.headers.get('cache-control'));
   console.log(response.headers.get('date'));
   console.log([...response.headers.keys()]);
   ```

   Analyze: what will each line log? DevTools Network displays both `date` and `server` in Response Headers. Why can client-side JavaScript not read them?
   Where this occurs: handling 429 status (Challenge 3). Documentation often suggests reading `Retry-After`, but for cross-origin targets scripts can only read it if exposed via `Access-Control-Expose-Headers`.
8. **HTTP cache.** Three identical requests. **Disable cache** in DevTools Network must be unchecked.

   ```
   await fetch('https://api.tvmaze.com/shows/1');
   await fetch('https://api.tvmaze.com/shows/1');
   await fetch('https://api.tvmaze.com/shows/1', { cache: 'no-store' });
   ```

   Analyze: which requests hit the network and which are served from cache? What does the Size column report? Which header from Experiment 7 controls this? If repeating the experiment, why is the first call served from cache?
   Where this occurs: revisiting a show (Challenge 2). Why does an application need in-memory cache if the browser already caches responses?
9. **Redirects.** `/lookup/shows` resolves a TVmaze show from an IMDb ID (see TVmaze docs, Show lookup).

   ```
   const response = await fetch('https://api.tvmaze.com/lookup/shows?
   imdb=tt0944947');
   console.log(response.status, response.redirected, response.url);
   console.log((await response.json()).name);
   ```

   Analyze: what will `status`, `redirected`, and `url` log? How many entries appear in DevTools Network and with what statuses?

Where this occurs: handling status codes in Challenge 3: client code never receives status 301 directly because `fetch` follows redirects transparently.

10. **Query strings.** Constructs two URLs without sending requests.

```
const query = 'Law & Order';
const plain = new URL(`https://api.tvmaze.com/search/shows?q=${query}`);
const encoded = new URL(`https://api.tvmaze.com/search/shows?${new
URLSearchParams({ q: query })}`);
console.log(plain.searchParams.get('q'),
[...plain.searchParams.keys()]);
console.log(encoded.searchParams.get('q'), encoded.search);
```

Analyze: what parameter value does the API receive in each case? How many search parameters exist in the first URL?
Where this occurs: search (Challenge 1): inserting "Law & Order" into a template literal without encoding truncates the query to "Law ".

4.3. CORS

Study:

- Cross-Origin Resource Sharing (CORS), Simple requests and Preflighted requests sections — MDN
- CORS-safelisted request header, Request: mode — MDN
- TVmaze documentation, CORS section

Investigate: origin structure; who grants access (server) vs. who enforces restrictions (browser); simple requests vs. `OPTIONS` preflight; meaning of `Access-Control-Allow-Origin: *`; effects of `mode: 'no-cors'`.

Experiments:

11. **Two TVmaze domains.** `api.tvmaze.com` is the API, `www.tvmaze.com` is the consumer website.

```
const api = await fetch('https://api.tvmaze.com/shows/82');
console.log('api:', api.status);
try {
  const site = await fetch('https://www.tvmaze.com/shows/82');
  console.log('www:', site.status);
} catch (error) {
  console.log('www:', error.name, error.message);
}
```

Analyze: what will each log output? Did the request reach `www.tvmaze.com` and did the server respond — what does DevTools Network show? What did the browser block: sending the request or reading the response?

Where this occurs: AI drafts occasionally query consumer web pages instead of API endpoints.

12. **CORS preflight.** The same API endpoint requested with `Content-Type` and with `Accept`.

```
try {
  const response = await fetch('https://api.tvmaze.com/shows/82', {
headers: { 'Content-Type': 'application/json' } });
  console.log('Content-Type:', response.status);
} catch (error) {
  console.log('Content-Type:', error.name, error.message);
}
const response = await fetch('https://api.tvmaze.com/shows/82', {
headers: { Accept: 'application/json' } });
console.log('Accept:', response.status);
```

Analyze: what will each line log? How many entries appear in Network for the first request, and with what HTTP method and status? TVmaze docs state: "All endpoints are CORS enabled." Why does the first request fail?
Where this occurs: search (Challenge 1). Adding `headers: { 'Content-Type': 'application/json' }` is a common habit from POST requests that AI agents frequently copy into GET calls.

13. **mode: 'no-cors'.** The same website request from Experiment 11, but using `mode: 'no-cors'`.

```
const response = await fetch('https://www.tvmaze.com/shows/82', { mode:
'no-cors' });
console.log(response.type, response.status, response.ok);
console.log(JSON.stringify(await response.text()));
```

Analyze: does an error occur? What values are logged for `type`, `status`, `ok`, and the response body?
Where this occurs: a deceptive CORS "fix" suggested by AI agents: the CORS error disappears, but the payload becomes inaccessible.

4.4. DevTools Network panel

Study: Inspect network activity, Network features reference (Filter, Throttling, Block requests, Override content, Timing) — Chrome DevTools.

Investigate using the experiments page or any website:

- Status, Type, Initiator, Size, Time, and Waterfall columns; meanings of `(canceled)`, `(failed)`, `(disk cache)`, and `CORS error`;
- Request tabs: Headers (General, Response, Request), Payload, Preview, Response, Initiator, Timing (Waiting for server response is TTFB);
- Throttling (Slow 4G, Offline), **Request blocking** (Ctrl+Shift+P → "Show Request blocking"), Preserve log, Copy → Copy as fetch;

- **Override content**: intercepting and modifying responses (e.g. returning HTML instead of JSON).

No dedicated table is required for this subsection: the Network panel is used in every challenge, and the Network checklist in the assignment (§6) is part of your report.

4.5. API documentation → `API.md`

Study the TVmaze API documentation: introduction (endpoints, embedding), Rate limiting, Caching, CORS, Licensing, and endpoints: Show search, Show main information, Show episode list, Show cast.

Task: Compile an `API.md` summary table to commit to your repository (§5.1) and use when drafting agent rules (assignment, §4). Verify every row using live DevTools Network requests, recording the exact request URL.

Question	What docs state	Actual request observations (URL)
Show search endpoint & query params		
Search response payload schema		
Show details endpoint; embedding episodes and cast in single call		
Nullable show fields (with example show ID)		
Empty search response		
Non-existent show ID response		
Rate limits & recommended 429 behavior		
Caching: what TVmaze caches vs. browser cache		
CORS: documentation promises vs. Experiment 12 findings		
Licensing & required attribution		

4.6. AI-generated asynchronous code patterns

Investigate recurring mistakes made by AI agents when writing fetch calls unprompted:

- Omitting `response.ok` checks, causing 404/500 responses to be treated as data or false "not found" results;
- Attaching `Content-Type: application/json` to GET requests;
- Firing requests on every keystroke or debouncing without aborting stale requests;
- Concatenating query parameters into template literals without encoding;
- Requesting show metadata, episodes, and cast sequentially, ignoring `embed`;
- Leaving loading states stuck on error (missing `finally`);
- Only logging errors to Console or using `alert()`;
- Treating aborted requests (`AbortError`) as errors;

- Retrying immediately in infinite loops on 429;
- Reading response headers that browsers restrict for cross-origin requests;
- Inventing non-existent endpoints or query parameters;
- "Fixing" CORS errors using `mode: 'no-cors'` or public CORS proxies.

Watch for these anti-patterns in AI drafts during §5. The agent rules file (assignment, §4) allows steering the agent against these habits upfront. Review how agents read instructions: Rules — Cursor, Adding repository custom instructions — GitHub Copilot.

5. Practical part: AI draft → audit → manual fixes

The **ShowFinder** starter project searches TV shows on TVmaze. In the completed app, the user types a query, browses results, selects a show, and views its details: description, episodes by season, and cast. HTML, CSS, UI state templates, and `src/view.js` rendering logic are already implemented. Only `src/api.js` (TVmaze API calls) and `src/app.js` (application state and flows) require implementation.

All practical tasks are in the assignment, §4: three sequential challenges, each with an AI prompt and numbered rules, with the `AGENTS.md` rules file created between Challenges 2 and 3. Keep the assignment open throughout. This section outlines your workflow: setup, AI interaction loop, and commit milestones.

Verification method: `check.html` tests compliance in the browser, grouped by challenge; rule numbers match test items (C3.2 tests Challenge 3 rule 2). It mocks TVmaze internally using recorded fixtures: simulating delays, returning 404/429/500, or severing connections. Thus, it operates independently of live connectivity or rate limits. Live TVmaze connectivity, real CORS errors, and DevTools Network behavior are verified via the Network checklist in the assignment, §6.

5.1. Setup

1. Download showfinder-starter.zip and extract it into a **separate repository** or into `lab-11/` of your shared coursework repository. Make the initial commit **without modifications**.
2. Add your completed `API.md` from §4.5 in a second commit: `API notes`.
3. Open `index.html` via Live Preview in Chrome or Edge, as in §4: verify the search form, empty results area, placeholder "Select a show from the search results.", and clean Console.
4. Open `check.html` similarly: confirm it reports "Passed: 0 of 24". Note the behaviors the automated suite does not cover.
5. Files `check.html`, `check/`, `offline.html`, and `src/view.js` must remain unmodified across all commits.

Classroom rate limits: The entire classroom goes online from one public IP address, and TVmaze guarantees only 20 requests per 10 seconds per address. If Network shows 429 status codes, switch to `offline.html` until rate limits reset: it runs the identical app against recorded fixtures. `check.html` never makes live network requests.

5.2. AI interaction guidelines

- **Research tasks (§4):** Write "My analysis" independently, before running snippets and without AI assistance.
- **Challenges:** Work through them sequentially: each builds on earlier steps. Follow this four-step loop (illustrated for Challenge 2):
  1. **AI draft.** Switch your IDE agent to editing mode (usually Agent mode). Close editor tabs for `check.html`, `check/`, and `API.md` so they are not included in prompt context. Submit **only** the challenge prompt from the assignment, verbatim. Do not paste the full assignment, `check.html`, or custom hints. The agent can consult TVmaze docs via the prompt link.
  2. **Commit draft.** Without manual edits, commit everything the agent changed using message `Challenge 2: AI draft`. Commit even if broken. The only exception is protected files (§5.1): if modified by the agent, revert them with `git restore check.html check offline.html src/view.js` and record this in your report.
  3. **Audit.** Open `check.html` to inspect the challenge block. For each failing test (*X*), identify which rule was violated and why, and reproduce it in `index.html`, gathering DevTools Network proof: status, headers, payload, `(canceled)`, initiator. Also inspect the git diff against constraints in assignment §5.
  4. **Manual fixes.** You may ask the AI to explain a test failure, rule, status, or header, but you **must write all fix code yourself**. When all checks in the current block and previous blocks display ✓, commit your fixes: `Challenge 2: fixes`.

  If an AI draft passes its entire block on the first attempt without breaking prior tests, omit the `fixes` commit and describe your verification method in your report. Agents often over-implement (e.g. handling errors in Challenge 1). Treat this as part of the draft and audit it within its respective challenge.

Example explanation prompt (ask / plan mode, without code editing):

> Explain why this test is failing. Here are the check name and failure message, the Network request details, and my code. Do not write the fix: identify the underlying fetch, HTTP, or CORS mechanism I overlooked and explain how to verify it in DevTools.

Include agent name, version, model, and date in your report.

5.3. Commit history

Required Git history:

1. Starter project unmodified;
2. `API notes`;
3. `Challenge 1: AI draft`, `Challenge 1: fixes`;
4. `Challenge 2: AI draft`, `Challenge 2: fixes`;
5. `Agent rules` — agent rules file (`AGENTS.md`);
6. `Challenge 3: AI draft`, `Challenge 3: fixes`.

If an AI draft passes cleanly without defects, omit the corresponding `fixes` commit (§5.2). On GitHub, each `fixes` commit demonstrates what you altered in the AI draft. At the defense, you use it to explain what was wrong in the draft (questions on your own code, §8).

After finishing Challenge 3, complete the Network checklist on your final release and deploy to GitHub Pages or Vercel (see instructions in starter `README.md`).

6. Report requirements

Email subject line:

```
[Web] Lab 11: AI agent for REST API integration
```

Include repository and deployed URLs — either GitHub Pages or Vercel:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

The report must contain:

1. Experiments table from §4 (all 13 experiments);
2. Links to `API.md` and `AGENTS.md` in the repository;
3. AI agent name, version, model, and session date;
4. The `check.html` result on the deployed site: "Passed: 24 of 24";
5. Completed Network checklist (N1–N6) with screenshots;
6. Summary of work: brief breakdown of each challenge, issues diagnosed in AI drafts, and fixes applied; note any protected files reverted if touched by the agent.

7. Oral defense

Defense is conducted as an interview based on your implementation and results. The questions below represent typical frontend developer interview topics. Base your answers

on your ShowFinder code, Section 4 experiments, and DevTools Network observations. You write the fixes, so the instructor may ask you to trace behavior in code, inspect DevTools live, or make code edits on the spot without AI assistance. Grading criteria: §8.

Study questions:

Fetch API

- What does `fetch()` return and when does its promise resolve?
- When does the `fetch` promise reject vs. resolve? Why do 404 or 500 responses not jump to `catch`?
- How does `response.ok` differ from `response.status`? For which status codes is `ok` true?
- Why can a response body only be read once?
- How do `response.json()` and `response.text()` differ? What occurs if the server returns HTML instead of JSON?
- How do you pass parameters to a GET request and why must they be encoded? Why is `URLSearchParams` preferable to template strings?
- Compare Fetch API, `XMLHttpRequest`, and third-party libraries like axios.

Promise and async/await

- What states can a Promise have? How does `then`/`catch` chaining compare to `async`/`await`?
- What does an async function return? What happens to unhandled exceptions thrown inside it?
- What distinguishes `return promise` from `return await promise` within a `try` block?
- What causes `Uncaught (in promise)` and how do you prevent it?
- Why does `forEach` with an async callback fail to wait for completion? How do you process arrays asynchronously?
- What is the purpose of `finally`? Provide a concrete loading state example.

Concurrent requests, aborting, debounce

- Contrast `Promise.all` with `Promise.allSettled`. When should you use each?
- How do you execute independent requests concurrently and why is it faster than sequential execution?
- What is a race condition in frontend development? How can stale responses overwrite newer ones and how is this avoided?
- How does `AbortController` function? What does calling code receive when a request is aborted, and why is this not treated as an application error?
- What is debounce and how does it differ from throttle? Why use debounce in search inputs?

HTTP and REST

- What is a REST API? What does GET idempotence mean?
- What do status codes 200, 301, 404, 429, 500, and 503 represent? Which are client errors and which are server errors?
- How should a client handle HTTP 429? What is backoff and the `Retry-After` header?
- What purposes do `Content-Type`, `Accept`, and `Cache-Control` serve? Which are sent by the client vs. server?
- How does `fetch` handle HTTP 301 redirects and how can client code detect them?
- What is resource embedding in REST APIs? When is a single call with `embed` preferable to multiple separate requests, and when is it not?

Caching

- How does browser HTTP caching operate? What does `Cache-Control: max-age=3600` mean?
- How does `cache: 'no-store'` in `fetch` change request handling?
- Why maintain an in-memory application cache when HTTP caching exists? Why must failed responses never be cached?

CORS

- What is the Same-Origin Policy and what components define an origin?
- What is CORS and who enforces it — browser or server?
- What qualifies as a simple request vs. requiring an `OPTIONS` preflight?
- Why can adding `Content-Type: application/json` break a GET request?
- Can a CORS error be resolved by client-side code? What does `mode: 'no-cors'` do and why is it not a fix?
- Why are certain response headers hidden from scripts when querying cross-origin endpoints?

UI states

- What states can an asynchronous data-fetching container have? How do you represent each?
- How does an empty result set differ from an error — in code and in user experience?
- What roles do `aria-busy` and `role="status"` play? How do screen readers announce search results?
- What information belongs in the user interface vs. developer Console on failure?

DevTools Network

- How do you identify which code line initiated a request?
- How do you simulate slow connections or offline environments?
- What do `(canceled)`, `(failed)`, `CORS error`, and `(disk cache)` indicate in Network?

- How does the Timing tab help distinguish server processing delays from network transfer delays?

API documentation

- What details do you inspect in API documentation prior to integration?
- Where did TVmaze documentation diverge from live API behavior, and how did you discover it?
- How do you handle nullable fields in response schemas?
- What is rate limiting and how does it affect client code?

AI assistants in software engineering

- Which asynchronous code bugs appeared most frequently in your AI drafts?
- How do you audit AI-generated code before incorporating it into a codebase?
- What is an agent rules file (`AGENTS.md`)? What rules did you define and how did they alter the Challenge 3 draft?

8. Grading

The lab is worth **3 points** (1 point each for research, practical implementation, and oral defense), plus **1 bonus point** for timely submission.

Component	Points	Criteria
Research (§4)	1	0.25 pts per subsection §4.1–§4.3: all experiments recorded in table with personal analysis and actual Console/Network findings, plus explanations where predictions differed (Report item 1). 0.25 pts for `API.md` (§4.5): all rows verified against live requests with URLs documented
Practical part (§5)	1	Self-check: "Passed: 24 of 24" in `check.html` on deployed site — 0.5 pts; 20 to 23 — 0.25 pts. Workflow: commits per §5.3, `AGENTS.md`, and Network checklist (Report item 5): all present and substantive — 0.5 pts; one of the three missing — 0.25 pts
Oral defense (§7)	1	Questions on your code — up to 0.5 pts; two questions from different groups in §7 — 0.25 pts each for complete answers
Bonus point	+1	Submission email sent on or before **22.10.2026**

> **Important!** Without completing the oral defense, combined research and practical points are capped at **1 point**: the defense confirms you understand the concepts rather than relying solely on AI output. The bonus point applies regardless of defense status.